Министерство науки и высшего образования Российской Федерации

Казанский национальный исследовательский технический университет – КАИ им. А.Н. Туполева

Институт компьютерных технологий и защиты информации Отделение

СПО ИКТЗИ «Колледж информационных технологий»

ТЕХНОЛОГИЯ РАЗРАБОТКИ И ЗАЩИТЫ БАЗ ДАННЫХ

Методические указания к лабораторным работам

Казань 2024

ОГЛАВЛЕНИЕ

ПРОЦЕСС СДАЧИ ВЫПОЛНЕННОЙ РАБОТЫ 2
ЛАБОРАТОРНАЯ РАБОТА №4 3
ПОДКЛЮЧЕНИЕ БД К WPF ПРИЛОЖЕНИЮ. ДОБАВЛЕНИЕ, ЧТЕНИЕ, РЕДАКТИРОВАНИЕ, УДАЛЕНИЕ ДАННЫХ ИЗ БД С ПОМОЩЬЮ WPF ПРИЛОЖЕНИЯ
ПРИЛОЖЕНИЕ А 16
ПРИЛОЖЕНИЕ Б..... 20

ПРОЦЕСС СДАЧИ ВЫПОЛНЕННОЙ РАБОТЫ

По итогам выполнения работы студент:

1. Демонстрирует преподавателю отчет о выполненной работе;
2. Демонстрирует приобретённые знания и навыки, отвечает на вопросы преподавателя по выполненному заданию, возможностям её доработки;
3. Демонстрирует отчет по выполненной лабораторной работе.

Итоговая оценка складывается из оценок по трем указанным составляющим.

ЛАБОРАТОРНАЯ РАБОТА №4

ПОДКЛЮЧЕНИЕ БД К WPF ПРИЛОЖЕНИЮ. ДОБАВЛЕНИЕ, ЧТЕНИЕ, РЕДАКТИРОВАНИЕ, УДАЛЕНИЕ ДАННЫХ ИЗ БД С ПОМОЩЬЮ WPF ПРИЛОЖЕНИЯ

ЦЕЛЬ

Написать WPF приложение, реализовать функции добавления, чтения, редактирования и удаления данных из БД.

ХОД РАБОТЫ

1. <u>База данных</u>

Перед тем, как приступить к разработке приложения, необходимо спроектировать и создать базу данных. Пример представлен на рисунке 1.1.

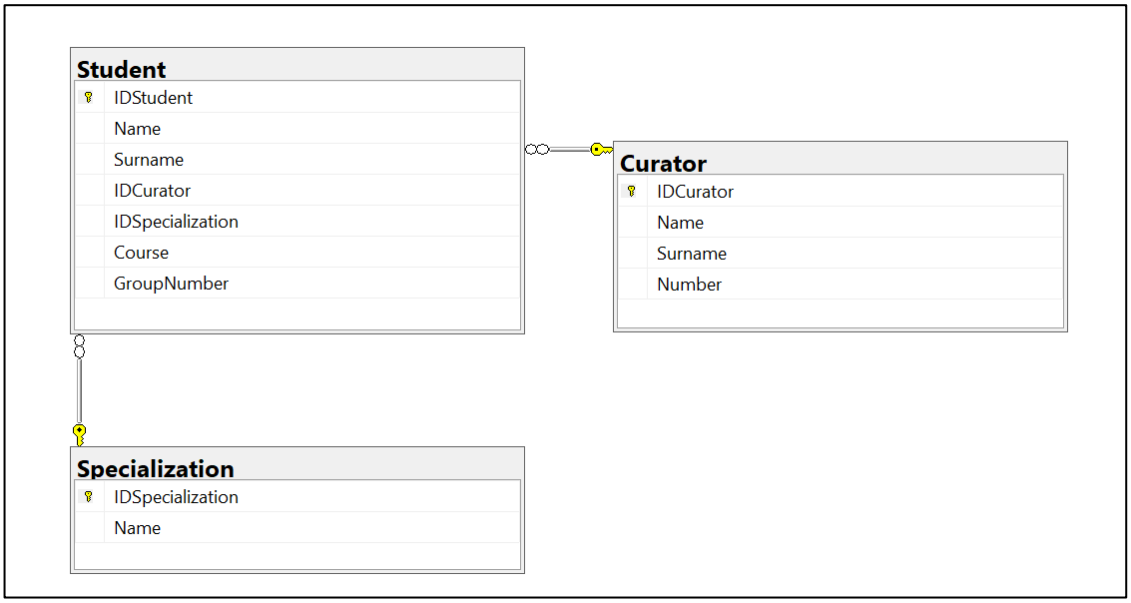

Рисунок 1.1 – ER-диаграмма

2. <u>Создание WPF приложения</u>

Windows Presentation Foundation (WPF) — аналог WinForms, система для построения клиентских приложений Windows с визуально привлекательными возможностями взаимодействия с пользователем, графическая (презентационная) подсистема в составе .NET Framework (начиная с версии 3.0), использующая язык XAML.

Открываем Visual Studio и выбираем шаблон «Приложение WPF (.NET Framework) (Рисунок 2.1). Далее называем наш проект и выбираем местоположение файлов.

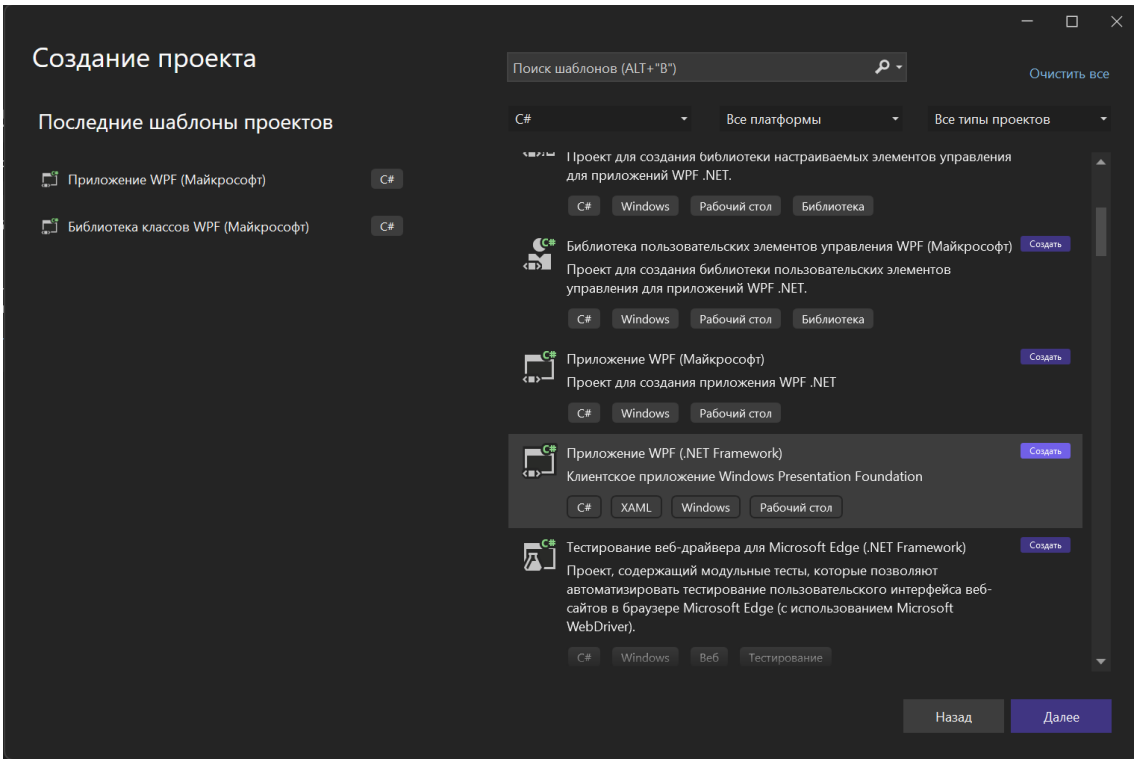

Рисунок 2.1 – Создание проекта

3. <u>Подключение пакетов</u>

Entity Framework представляет специальную объектно-ориентированную технологию на базе фреймворка .NET для работы с данными.

Шаг 1. Для начала добавим пакет NuGet для Entity Framework 6. В Обозреватель решений выберите узел проекта. В главном меню выберите “Управление пакетами NuGet” (Рисунки 3.1 – 3.2).

Шаг 2. В диспетчер пакетов NuGet щелкните ссылку “Обзор”, найдите “EntityFramework” и нажмите кнопку "Установить" в правой области и следуйте инструкциям. Окно вывода сообщает о завершении установки.

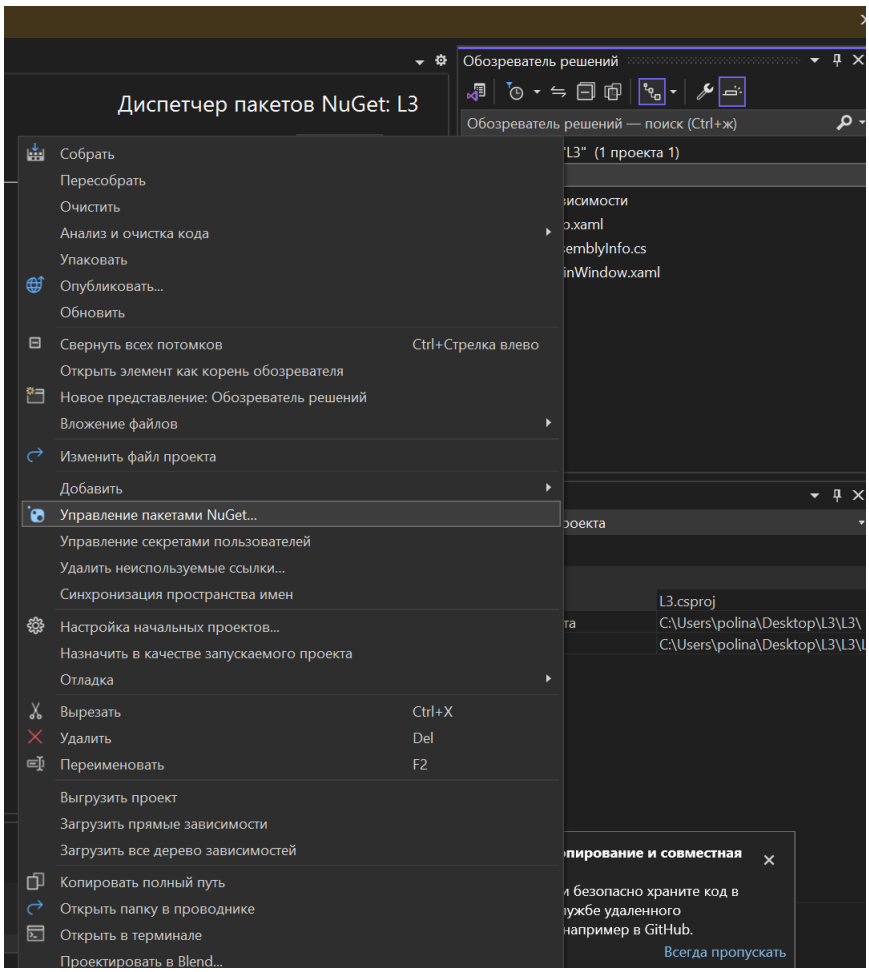

Рисунок 3.1 – Управление пакетами NuGet

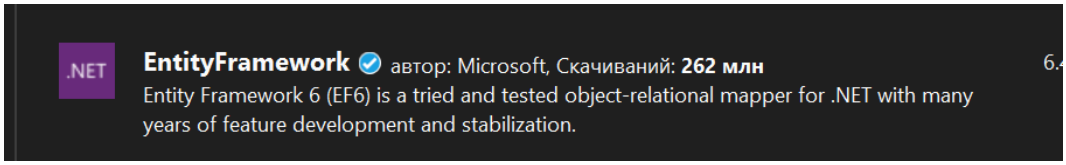

Рисунок 3.2 – Пакет «EntityFramework»

4. Создание модели ADO.NET

Шаг 1. Нажмём правой кнопкой «мыши» по нашему проекту в обозревателе решений. Далее необходимо добавить новый элемент. Для этого выберем «Добавить» (Рисунок 4.1).

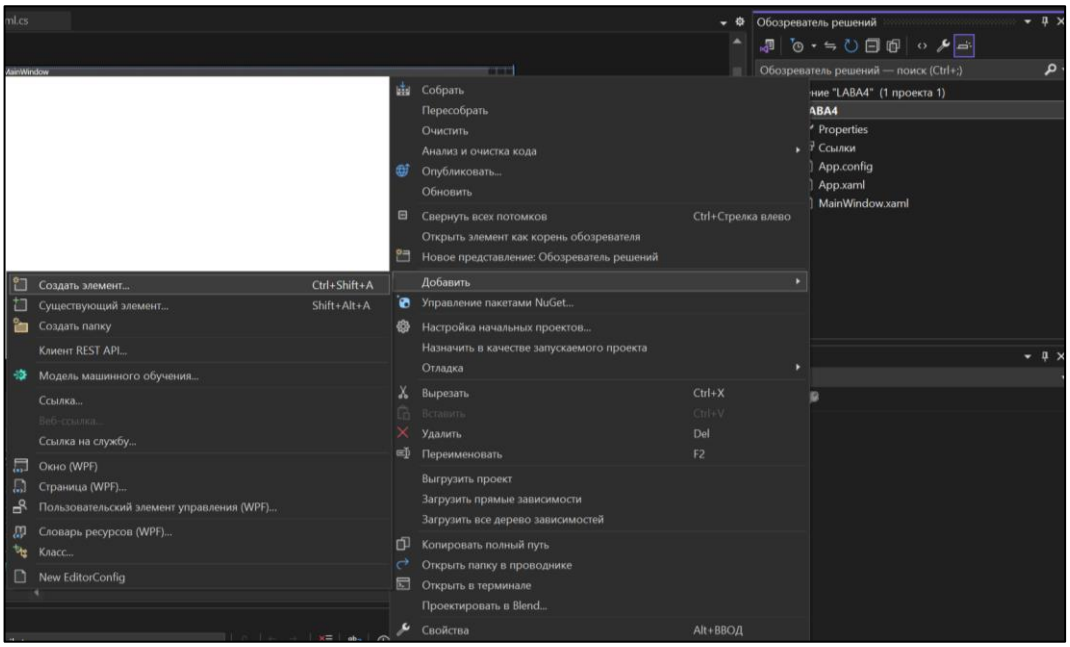

Рисунок 4.1 – Добавление нового элемента

Шаг 2. Выберем элемент «Модель ADO.NET EDM», назовем его и нажмём кнопку «Добавить» (Рисунок 4.2).

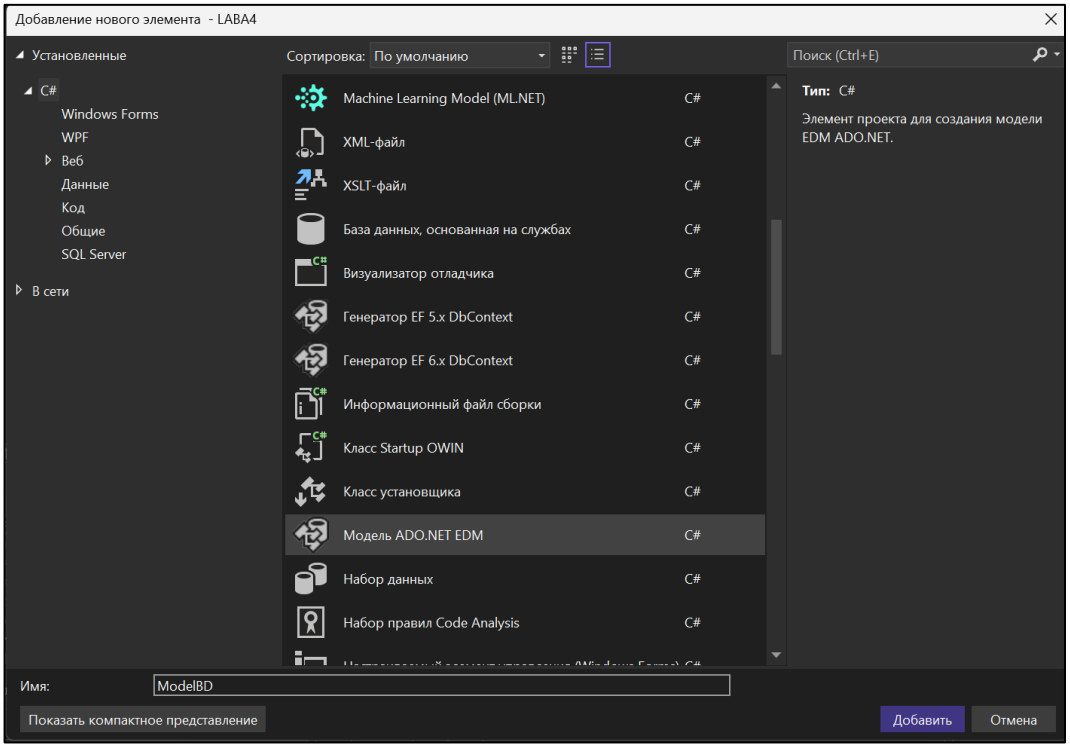

Рисунок 4.2 – Выбор нужного элемента

Шаг 3. Далее откроется Мастер моделей EDM, выберем «Конструктор EF из базы данных» и нажмём «Далее» (Рисунок 4.3).

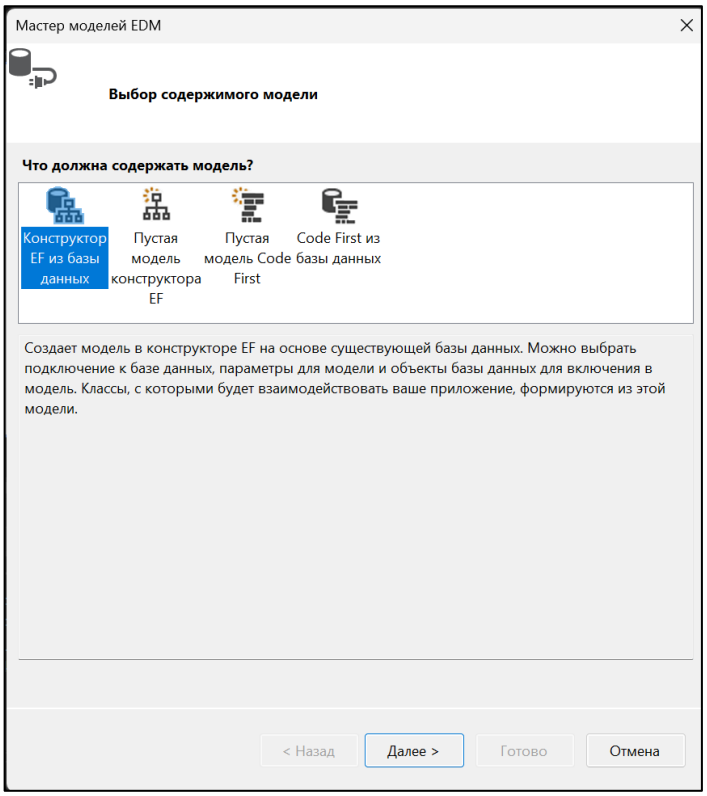

Рисунок 4.3 – Выбор содержимого модели

Шаг 4. Теперь необходимо создать соединение (Рисунок 4.4).

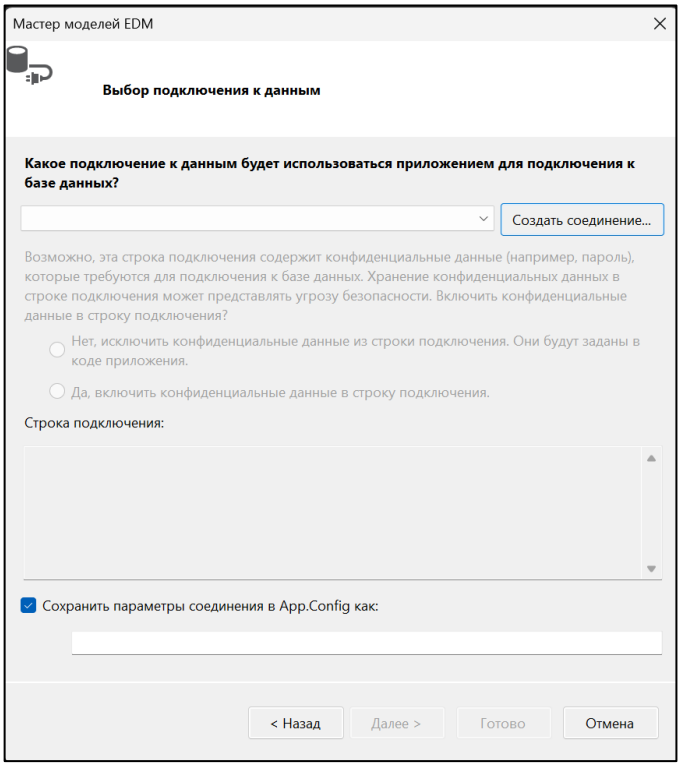

Рисунок 4.4 – Выбор подключения к данным

Шаг 5. Выберем источник данных «Microsoft SQL Server» и нажмём кнопку «Продолжить» (Рисунок 4.5).

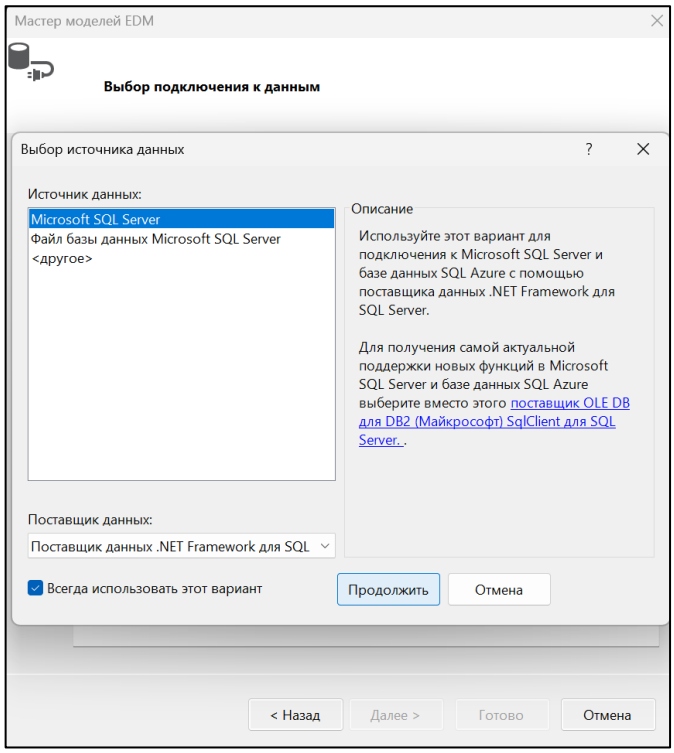

Рисунок 4.5 – Выбор источника данных

Шаг 6. Пропишем имя сервера и выберем имя нужной нам базы данных и нажмём на кнопку «Ок» (Рисунок 4.6).

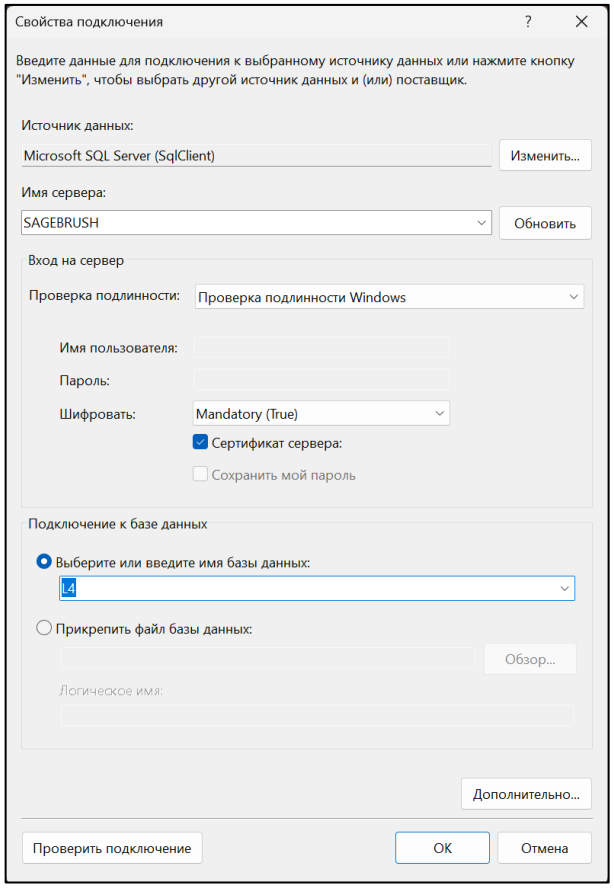

Рисунок 4.6 – Свойства подключения

Шаг 7. Пропишем название параметра соединения (Рисунок 4.7).

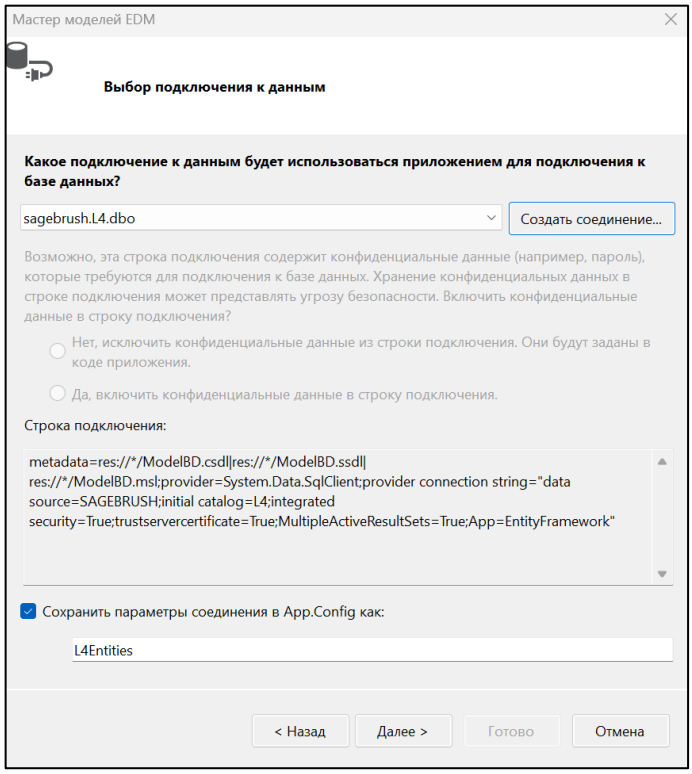

Рисунок 4.7 – Выбор подключения к данным

Шаг 8. Выберем версию Entity Framework (Рисунок 4.8).

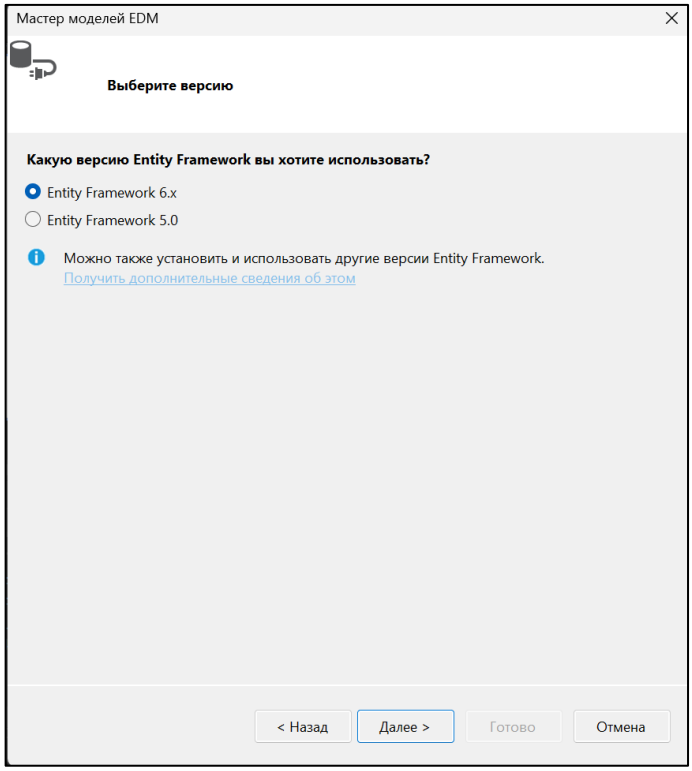

Рисунок 4.8 – Выбор версии

Шаг 9. Теперь необходимо выбрать объекты базы данных, которые вы

хотите включить в модель. Выберем только таблицы (Рисунок 4.9). Нажмем кнопку «Готово».

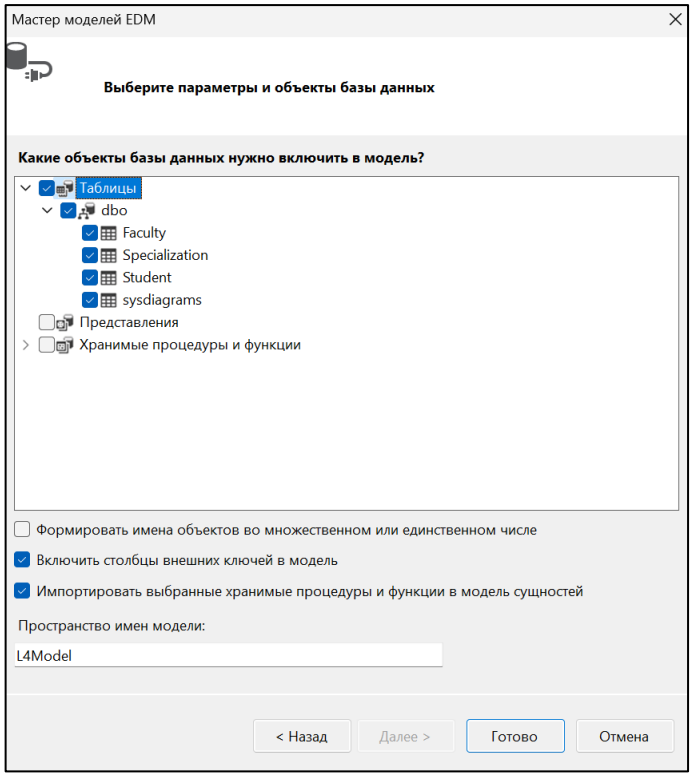

Рисунок 4.9 – Выбор параметров и объектов базы данных

Шаг 10. Модель готова (Рисунок 4.10).

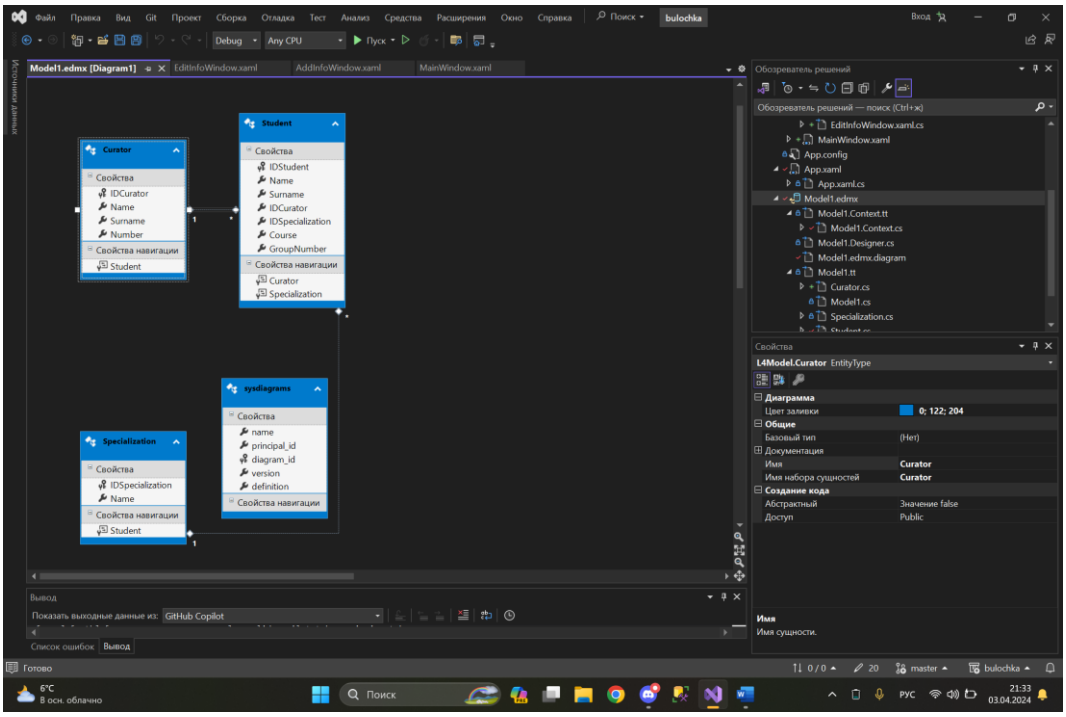

Рисунок 4.10 – Модель EDMX

5. Обращение к модели данных

Шаг 1. Открываем файл НазваниеВашейМодели.Context.cs (Рисунок 5.1) и добавляем приватное статичное поле, которое будет контекстом:

```
private static L4Entities _context;
```

Шаг 2. Добавляем метод получения экземпляра этого контекста:

```
public static L4Entities GetContext()
 {
     if (_context == null)
         _context = new L4Entities();
     return _context;
 }
```

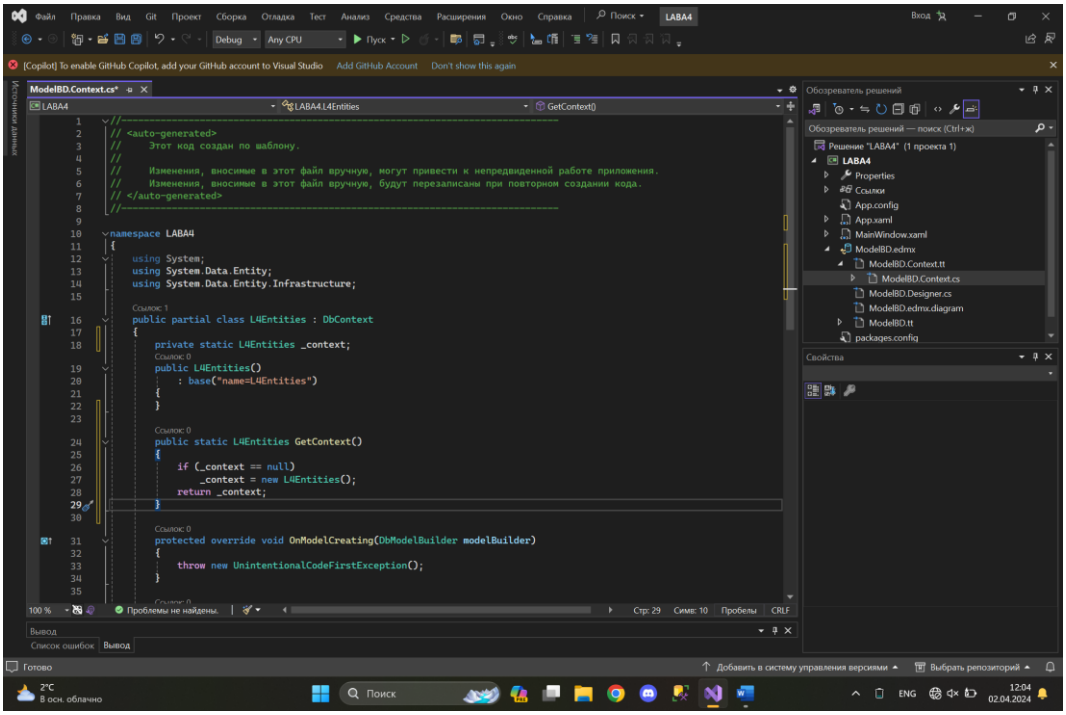

Рисунок 5.1 – Контекст данных

Шаг 3. Для отображения, добавления, изменения и удаления данных создадим окна, как представлено на рисунках 5.2–5.4. Теперь необходимо написать разметку наших окон. Листинг разметки представлен в ПРИЛОЖЕНИИ А.

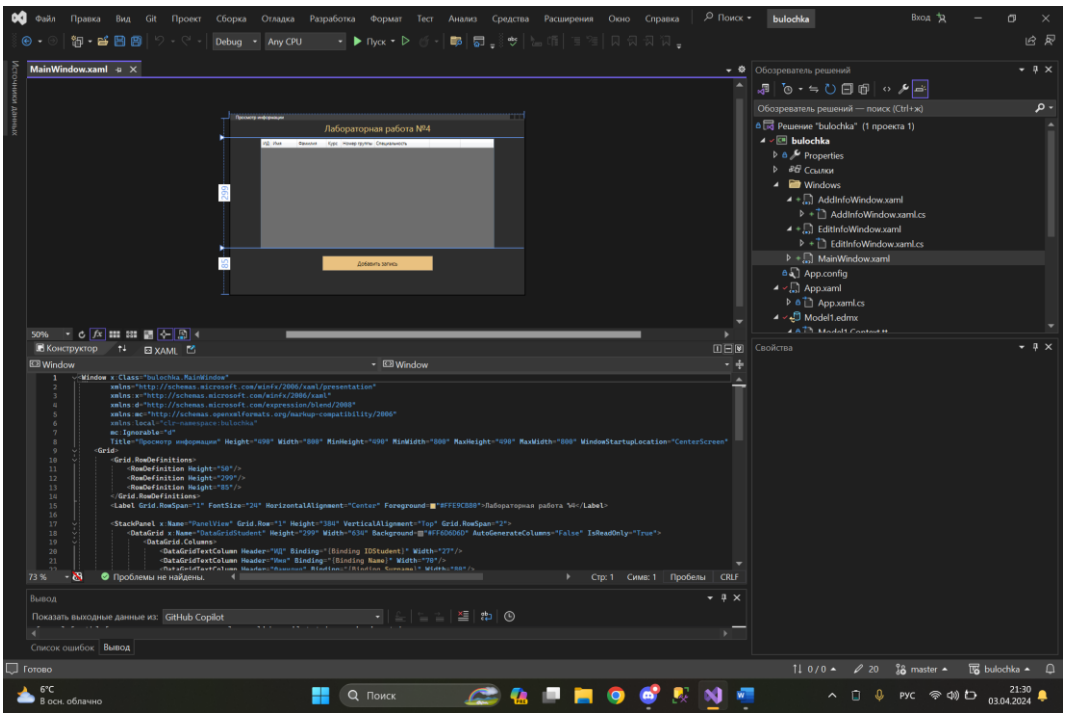

Рисунок 5.2 – Окно «MainWindow»

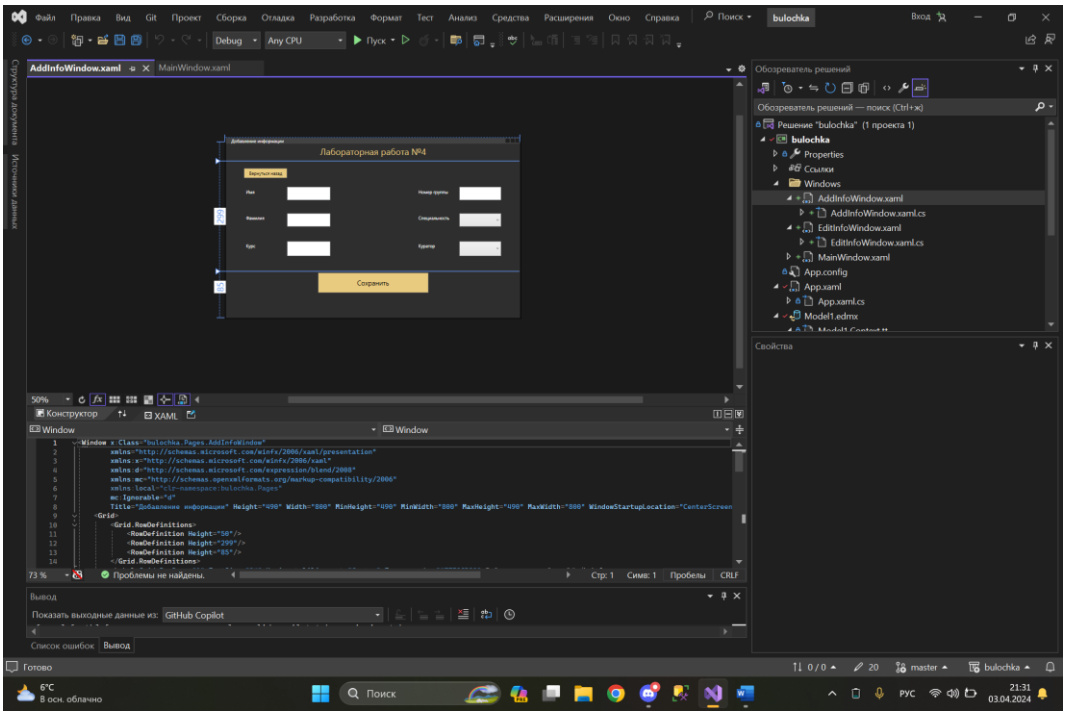

Рисунок 5.3 – Окно «AddInfoWindow»

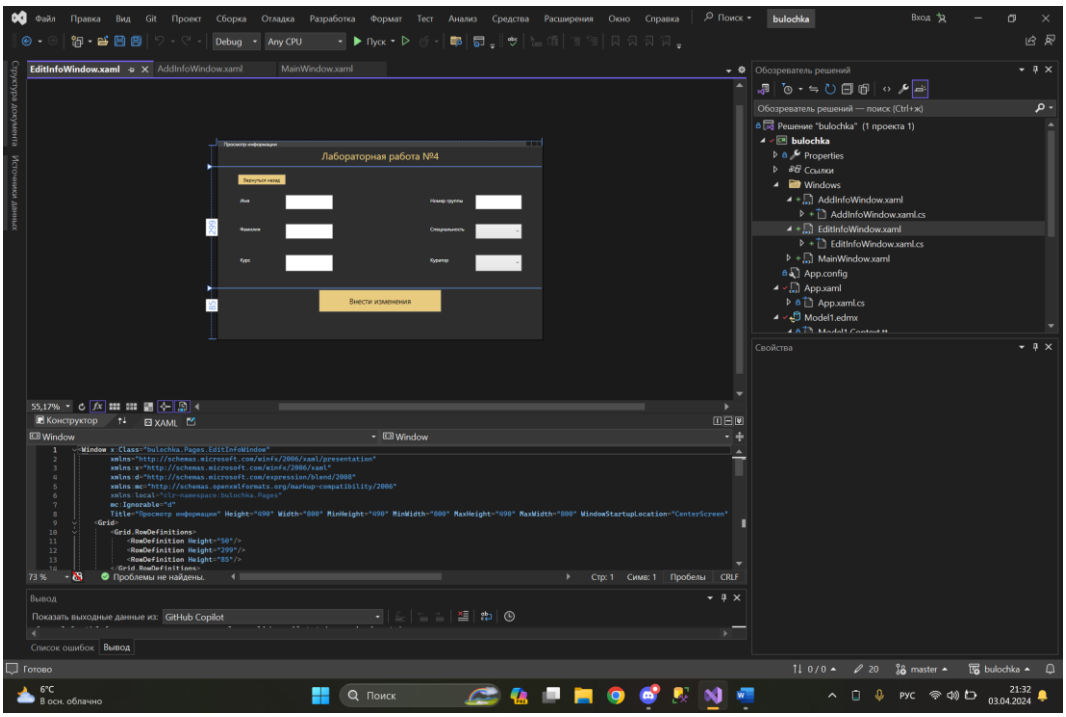

Рисунок 5.4 – Окно «EditInfoWindow»

Шаг 4. Для вывода данных из БД будем использовать элемент DataGrid. Список данных выводится по столбцам. Устанавливаем их с помощью свойства DataGrid.Columns, которое и описывает набор столбцов:

- DataGridTextColumn для текстовых столбцов;
- DataGridTemplateColumn для более сложного представления данных, например, кнопки.

Привязка столбцов осуществляется с помощью атрибута Binding:

```
<DataGridTextColumn Header="ИД" Binding="{Binding IDStudent}" Width="27"/>
```

Шаг 5. Для того, чтобы вывести данные в таблицу, необходимо нажать на F7 и обратиться к контексту модели:

```
DataGridStudent.ItemsSource = L4Entities.GetContext().Student.ToList();
```

Шаг 6. Доработка элемента «ComboBox»: как и всем другим используемым элементам, необходимо дать ему имя, а также указать отображаемое свойство и осуществить привязку к необходимому столбцу нашей сущности:

```
<ComboBox x:Name="ComboBoxSpecialization" SelectedItem="{Binding Specialization}"
Grid.Row="5" Grid.Column="5" DisplayMemberPath="Name"/>
```

Загрузим список данных с помощью следующей команды:

```
ComboBoxSpecialization.ItemsSource = L4Entities.GetContext().Specialization.ToList();
```

Шаг 7. После того, как наша разметка будет готова, а нужные элементы привязаны к столбцам, приступим к логике взаимодействия. Код представлен в ПРИЛОЖЕНИИ Б.

На странице добавим новое поле, которое будет хранить в себе экземпляр добавляемого студента:

```
private Student _currentStudent = new Student();
```

При инициализации установим DataContext страницы — этот созданный объект:

```
DataContext = _currentStudent;
```

Функция добавления. Обработаем нажатие на кнопку «Сохранить», в коде пропишем логику обращения к модели данных и добавления нового экземпляра студента, а также сохранение изменений в БД:

```
L4Entities.GetContext().Student.Add(_currentStudent);
L4Entities.GetContext().SaveChanges();
```

Для обновления данных в элементе DataGrid пропишем следующую команду:

```
DataGridStudent.ItemsSource = L4Entities.GetContext().Student.OrderBy(x =>
x.IDStudent).ToList();
```

Функция редактирования. В первую очередь добавим параметр нашей странице. В нее мы будем передавать экземпляр выбранного студента и, в случае если он не пустой, присваивать нашему полю CurrentStudent:

```
public EditInfoWindow(Student selectStudent)
 {
     InitializeComponent();
     if(selectStudent != null)
     {
         _currentStudent = selectStudent;
         DataContext = _currentStudent;
     }
 }
```

Для кнопки «Сохранить изменения» пропишем следующую команду:

```
L4Entities.GetContext().SaveChanges();
```

Функция удаления. Для реализации функции удаления отдельное окно не потребуется. Создадим диалоговое окно, запрашивающее подтверждение нашего действия, и если результатом диалога от пользователя было нажатие на кнопку «yes», то мы будем выполнять удаление.

Для удаления экземпляра студента и сохранения изменений в БД пропишем следующие команды:

```
var StudentRemoving = DataGridStudent.SelectedItems.Cast<Student>().ToList();

L4Entities.GetContext().Student.RemoveRange(StudentRemoving);
L4Entities.GetContext().SaveChanges();
```

7. Задание на лабораторную работу:

1) ознакомиться с теоретической частью лабораторной работы;
2) создать WPF приложение по вашей теме курсовой работы и подключить базу данных;
3) осуществить следующие функции: вывод, добавление, удаление и изменение данных;
4) оформить отчет по результатам выполненной работы.

Критерии оценивания:

- на оценку «3»: реализовать все необходимые функции для 2-х сущностей;
- на оценку «4»: реализовать все необходимые функции минимум для 50% сущностей;
- на оценку «5»: реализовать все необходимые функции минимум для 70% сущностей.

А также при оценивании работы будет учитываться внешний вид приложения, а так же обработка исключений.

Данная программа является лишь примером того, как можно выполнить задание.

ПРИЛОЖЕНИЕ А

РАЗМЕТКА

Основное окно “MainWindow”:

```
<Window x:Class="bulochka.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        xmlns:local="clr-namespace:bulochka"
        mc:Ignorable="d"
        Title="Просмотр информации" Height="490" Width="800" MinHeight="490"
MinWidth="800" MaxHeight="490" MaxWidth="800" WindowStartupLocation="CenterScreen"
Background="#FF2E2E2E">
    <Grid>
        <Grid.RowDefinitions>
            <RowDefinition Height="50"/>
            <RowDefinition Height="299"/>
            <RowDefinition Height="85"/>
        </Grid.RowDefinitions>
        <Label Grid.RowSpan="1" FontSize="24" HorizontalAlignment="Center"
Foreground="#FFE9CB80">Лабораторная работа №4</Label>

        <StackPanel x:Name="PanelView" Grid.Row="1" Height="384"
VerticalAlignment="Top" Grid.RowSpan="2">
            <DataGrid x:Name="DataGridStudent" Height="299" Width="634"
Background="#FF6D6D6D" AutoGenerateColumns="False" IsReadOnly="True">
                <DataGrid.Columns>
                    <DataGridTextColumn Header="ИД" Binding="{Binding IDStudent}"
Width="27"/>
                    <DataGridTextColumn Header="Имя" Binding="{Binding Name}"
Width="70"/>
                    <DataGridTextColumn Header="Фамилия" Binding="{Binding Surname}"
Width="80"/>
                    <DataGridTextColumn Header="Курс" Binding="{Binding Course}"
Width="40"/>
                    <DataGridTextColumn Header="Номер группы" Binding="{Binding
GroupNumber}"/>
                    <DataGridTextColumn Header="Специальность" Binding="{Binding
Specialization.Name}" Width="150"/>
                    <DataGridTemplateColumn Width="84">
                        <DataGridTemplateColumn.CellTemplate>
                            <DataTemplate>
                                <Button x:Name="ButtonDeleteGrid" Content="Удалить"
Background="#FFE9CB80" Click="ButtonDeleteGrid_Click"/>
                            </DataTemplate>
                        </DataGridTemplateColumn.CellTemplate>
                    </DataGridTemplateColumn>
                    <DataGridTemplateColumn Width="84">
                        <DataGridTemplateColumn.CellTemplate>
                            <DataTemplate>
                                <Button x:Name="ButtonEditGrid" Content="Изменить"
Background="#FFE9CB80" Click="ButtonEditGrid_Click"/>
                            </DataTemplate>
                        </DataGridTemplateColumn.CellTemplate>
                    </DataGridTemplateColumn>
                </DataGrid.Columns>
            </DataGrid>
            <Button x:Name="ButtonAdd" Height="40" Width="300" Content="Добавить
запись" FontSize="14" Background="#FFE9C180" Click="ButtonAdd_Click"
Margin="0,20,0,0"/>
        </StackPanel>
    </Grid>
</Window>
```

Окно для добавления информации “AddInfoWindow”:

```
<Window x:Class="bulochka.Pages.AddInfoWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        xmlns:local="clr-namespace:bulochka.Pages"
        mc:Ignorable="d"
        Title="Добавление информации" Height="490" Width="800" MinHeight="490"
MinWidth="800" MaxHeight="490" MaxWidth="800" WindowStartupLocation="CenterScreen"
Background="#FF2E2E2E">
    <Grid>
        <Grid.RowDefinitions>
            <RowDefinition Height="50"/>
            <RowDefinition Height="299"/>
            <RowDefinition Height="85"/>
        </Grid.RowDefinitions>
        <Label Grid.RowSpan="1" FontSize="24" HorizontalAlignment="Center"
Foreground="#FFE9CB80">Лабораторная работа №4</Label>

        <StackPanel x:Name="PanelAdd" Grid.Row="1" Visibility="Visible"
Grid.RowSpan="2">
            <Grid x:Name="GridAdd" Height="301">
                <Grid.RowDefinitions>
                    <RowDefinition Height="17*"/>
                    <RowDefinition Height="26*"/>
                    <RowDefinition Height="25*"/>
                    <RowDefinition Height="35*"/>
                    <RowDefinition Height="36*"/>
                    <RowDefinition Height="37*"/>
                    <RowDefinition Height="39*"/>
                    <RowDefinition Height="40*"/>
                    <RowDefinition Height="46*"/>
                </Grid.RowDefinitions>
                <Grid.ColumnDefinitions>
                    <ColumnDefinition Width="50*"/>
                    <ColumnDefinition Width="117*"/>
                    <ColumnDefinition Width="117*"/>
                    <ColumnDefinition Width="235*"/>
                    <ColumnDefinition Width="117*"/>
                    <ColumnDefinition Width="113*"/>
                    <ColumnDefinition Width="52*"/>
                </Grid.ColumnDefinitions>
                <Label Grid.Row="3" Grid.Column="1" Foreground="White"
Grid.ColumnSpan="2">Имя</Label>
                <Label Grid.Row="5" Grid.Column="1" Foreground="White"
Grid.ColumnSpan="2">Фамилия</Label>
                <Label Grid.Row="7" Grid.Column="1" Foreground="White"
Grid.ColumnSpan="2">Курс</Label>
                <Label Grid.Row="3" Grid.Column="4" Foreground="White"
Grid.ColumnSpan="2">Номер группы</Label>
                <Label Grid.Row="5" Grid.Column="4" Foreground="White"
Grid.ColumnSpan="2">Специальность</Label>
                <Label Grid.Row="7" Grid.Column="4" Foreground="White"
Grid.ColumnSpan="2">Куратор</Label>
                <TextBox x:Name="TextBoxName" Text="{Binding Name}" Grid.Row="3"
Grid.Column="2" PreviewTextInput="TextBoxName_PreviewTextInput" MaxLength="20"/>
                <TextBox x:Name="TextBoxSurname" Text="{Binding Surname}" Grid.Row="5"
Grid.Column="2" PreviewTextInput="TextBoxName_PreviewTextInput" MaxLength="20"/>
                <TextBox x:Name="TextBoxCourse" Text="{Binding Course}" Grid.Row="7"
Grid.Column="2" PreviewTextInput="TextBoxCourse_PreviewTextInput" MaxLength="20"/>
                <TextBox x:Name="TextBoxGroupNumber" Text="{Binding GroupNumber}"
Grid.Row="3" Grid.Column="5" PreviewTextInput="TextBoxCourse_PreviewTextInput"
MaxLength="20"/>
                <ComboBox x:Name="ComboBoxSpecialization" SelectedItem="{Binding
Specialization}" Grid.Row="5" Grid.Column="5" DisplayMemberPath="Name"/>
                <ComboBox x:Name="ComboBoxCurator" SelectedItem="{Binding Curator}"
Grid.Row="7" Grid.Column="5" DisplayMemberPath="Surname"/>
```

```
                <Button x:Name="ButtonAddBack" Grid.Row="1" Grid.Column="1"
Content="Вернуться назад" Background="#FFE9CB80" Click="ButtonAddBack_Click"
Grid.ColumnSpan="2" Height="26" Margin="0,0,117,0"
VerticalAlignment="Bottom"></Button>
            </Grid>
            <Button Grid.Row="2" x:Name="ButtonSave" Height="55" Width="300"
Content="Сохранить" FontSize="18" Background="#FFE9CB80" Click="ButtonSave_Click"/>
        </StackPanel>
    </Grid>
</Window>
```

Окно для изменения информации “EditInfoWindow”:

```
<Window x:Class="bulochka.Pages.EditInfoWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        xmlns:local="clr-namespace:bulochka.Pages"
        mc:Ignorable="d"
        Title="Просмотр информации" Height="490" Width="800" MinHeight="490"
MinWidth="800" MaxHeight="490" MaxWidth="800" WindowStartupLocation="CenterScreen"
Background="#FF2E2E2E" IsVisibleChanged="Window_IsVisibleChanged">
    <Grid>
        <Grid.RowDefinitions>
            <RowDefinition Height="50"/>
            <RowDefinition Height="299"/>
            <RowDefinition Height="85"/>
        </Grid.RowDefinitions>
        <Label Grid.RowSpan="1" FontSize="24" HorizontalAlignment="Center"
Foreground="#FFE9CB80">Лабораторная работа №4</Label>

        <StackPanel x:Name="PanelEdit" Grid.Row="1" Grid.RowSpan="2">
            <Grid x:Name="GridEdit" Height="301">
                <Grid.RowDefinitions>
                    <RowDefinition Height="17*"/>
                    <RowDefinition Height="26*"/>
                    <RowDefinition Height="25*"/>
                    <RowDefinition Height="35*"/>
                    <RowDefinition Height="36*"/>
                    <RowDefinition Height="37*"/>
                    <RowDefinition Height="39*"/>
                    <RowDefinition Height="40*"/>
                    <RowDefinition Height="46*"/>
                </Grid.RowDefinitions>
                <Grid.ColumnDefinitions>
                    <ColumnDefinition Width="50*"/>
                    <ColumnDefinition Width="117*"/>
                    <ColumnDefinition Width="117*"/>
                    <ColumnDefinition Width="119*"/>
                    <ColumnDefinition Width="116*"/>
                    <ColumnDefinition Width="117*"/>
                    <ColumnDefinition Width="113*"/>
                    <ColumnDefinition Width="52*"/>
                </Grid.ColumnDefinitions>
                <Label Grid.Row="3" Grid.Column="1" Foreground="White">Имя</Label>
                <Label Grid.Row="5" Grid.Column="1" Foreground="White">Фамилия</Label>
                <Label Grid.Row="7" Grid.Column="1" Foreground="White">Курс</Label>
                <Label Grid.Row="3" Grid.Column="5" Foreground="White">Номер
группы</Label>
                <Label Grid.Row="5" Grid.Column="5"
Foreground="White">Специальность</Label>
                <Label Grid.Row="7" Grid.Column="5" Foreground="White">Куратор</Label>
                <TextBox x:Name="TextBoxEditName" Text="{Binding Name}" Grid.Row="3"
Grid.Column="2"  PreviewTextInput="TextBoxEditName_PreviewTextInput" MaxLength="20"/>
                <TextBox x:Name="TextBoxEditSurname" Text="{Binding Surname}"
Grid.Row="5" Grid.Column="2" PreviewTextInput="TextBoxEditName_PreviewTextInput"
MaxLength="20"/>
```

```
                <TextBox x:Name="TextBoxEditCourse" Text="{Binding Course}"
Grid.Row="7" Grid.Column="2" PreviewTextInput="TextBoxEditCourse_PreviewTextInput"
MaxLength="20"/>
                <TextBox x:Name="TextBoxEditGroupNumber" Text="{Binding GroupNumber}"
Grid.Row="3" Grid.Column="6"  PreviewTextInput="TextBoxEditCourse_PreviewTextInput"
MaxLength="20"></TextBox>
                <Button x:Name="ButtonEditBack" Grid.Row="1" Grid.Column="1"
Content="Вернуться назад" Background="#FFE9CB80" Click="ButtonEditBack_Click"
></Button>
                <ComboBox x:Name="ComboBoxEditSpecialization" SelectedItem="{Binding
Specialization}" Grid.Row="5" Grid.Column="6" DisplayMemberPath="Name"/>
                <ComboBox x:Name="ComboBoxEditCurator" SelectedItem="{Binding
Curator}" Grid.Row="7" Grid.Column="6" DisplayMemberPath="Surname"/>
            </Grid>
            <Button x:Name="ButtonSaveEdit" Height="55" Width="300" Content="Внести
изменения" FontSize="18" Background="#FFE9CB80" Click="ButtonEdit_Click"/>
        </StackPanel>
    </Grid>

</Window>
```

ПРИЛОЖЕНИЕ Б

ЛОГИКА ВЗАИМОДЕЙСТВИЯ

Основное окно “MainWindow”:

```
public partial class MainWindow : Window
{
    private Student _currentStudent = new Student();

    public MainWindow()
    {
        InitializeComponent();
        DataContext = _currentStudent;
        DataGridStudent.ItemsSource = L4Entities.GetContext().Student.ToList();
        DataGridStudent.ItemsSource = L4Entities.GetContext().Student.OrderBy(x =>
x.IDStudent).ToList();
    }
    private void ButtonAdd_Click(object sender, RoutedEventArgs e)
    {
        new AddInfoWindow().Show();
        Close();
    }

    private void ButtonDeleteGrid_Click(object sender, RoutedEventArgs e)
    {
        var studentRemoving = DataGridStudent.SelectedItems.Cast<Student>().ToList();
        if (MessageBox.Show($"Подтвердить удаление?", "!",
                MessageBoxButton.YesNo, MessageBoxImage.Question) !=
MessageBoxResult.Yes) return;

        L4Entities.GetContext().Student.RemoveRange(studentRemoving);
        L4Entities.GetContext().SaveChanges();
        MessageBox.Show("Ура! Удалилось!");
        DataGridStudent.ItemsSource = L4Entities.GetContext().Student.OrderBy(x =>
x.IDStudent).ToList();
    }

    private void ButtonEditGrid_Click(object sender, RoutedEventArgs e)
    {
        var item = (Student)DataGridStudent.SelectedItem;
        new EditInfoWindow(item).Show();
        Close();
    }
}
```

Окно для добавления информации “AddInfoWindow”:

```
public partial class AddInfoWindow : Window
{public partial class AddInfoWindow : Window
{
    private readonly Student _currentStudent = new Student();
    private readonly L4Entities _bd = new L4Entities();

    public AddInfoWindow()
    {
        InitializeComponent();
        DataContext = _currentStudent;
        ComboBoxSpecialization.ItemsSource =
L4Entities.GetContext().Specialization.ToList();
        ComboBoxCurator.ItemsSource = L4Entities.GetContext().Curator.ToList();
    }

    private void ButtonSave_Click(object sender, RoutedEventArgs e)
    {
        var flag = true;
        foreach (Control c in GridAdd.Children)
```

```
        {
            if (c.GetType() == typeof(TextBox))
            {
                if (((TextBox)c).Text == String.Empty)
                {
                    flag = false;
                }
            }

            if (c.GetType() == typeof(ComboBox))
            {
                if (((ComboBox)c).Text == String.Empty)
                {
                    flag = false;
                }
            }
        }
        if (flag) {
            L4Entities.GetContext().Student.Add(_currentStudent);
            L4Entities.GetContext().SaveChanges();
            MessageBox.Show("Ура! Добавилось!");
            new MainWindow().Show();
            Close();
        }
        else MessageBox.Show("Пусто и грустно :( ");
    }

    private void ButtonAddBack_Click(object sender, RoutedEventArgs e)
    {
        new MainWindow().Show();
        Close();
    }

    private void TextBoxCourse_PreviewTextInput(object sender,
TextCompositionEventArgs e)
    {
        if (!char.IsDigit(e.Text, 0))
        {
            e.Handled = true;
        }
    }

    private void TextBoxName_PreviewTextInput(object sender, TextCompositionEventArgs
e)
    {
        foreach (char c in e.Text)
        {
            if (!char.IsLetter(c))
            {
                e.Handled = true;
                break;
            }
        }
    }
}
```

Окно для изменения информации “EditInfoWindow”:

```
public partial class EditInfoWindow : Window
{
    private Student _currentStudent = new Student();
    L4Entities bd = new L4Entities();

    public EditInfoWindow(Student selectStudent)
    {
        InitializeComponent();
        if (selectStudent == null) return;
```

```
        _currentStudent = selectStudent;
        DataContext = _currentStudent;
        ComboBoxEditSpecialization.ItemsSource =
L4Entities.GetContext().Specialization.ToList();
        ComboBoxEditCurator.ItemsSource = L4Entities.GetContext().Curator.ToList();
    }
    private void ButtonEdit_Click(object sender, RoutedEventArgs e)
    {
        var flag = true;
        foreach (Control c in GridEdit.Children)
        {
            if (c.GetType() == typeof(TextBox))
            {
                if (((TextBox)c).Text == String.Empty)
                {
                    flag = false;
                }
            }

            if (c.GetType() == typeof(ComboBox))
            {
                if (((ComboBox)c).Text == String.Empty)
                {
                    flag = false;
                }
            }
        }
        if (flag)
        {
            L4Entities.GetContext().SaveChanges();
            MessageBox.Show("Ура! Изменилось!");
            new MainWindow().Show();
            Close();
        }
        else MessageBox.Show("Пусто и грустно :( ");

    }
    private void ButtonEditBack_Click(object sender, RoutedEventArgs e)
    {
        new MainWindow().Show();
        Close();
    }

    private void TextBoxEditCourse_PreviewTextInput(object sender,
TextCompositionEventArgs e)
    {
        if (!char.IsDigit(e.Text, 0))
        {
            e.Handled = true;
        }
    }

    private void TextBoxEditName_PreviewTextInput(object sender,
TextCompositionEventArgs e)
    {
        foreach (char c in e.Text)
        {
            if (!char.IsLetter(c))
            {
                e.Handled = true;
                break;
            }
        }
    }
}
```